

Wichtige Befehle für Kommunikation

Im nachfolgenden Kapitel werden in Abhängigkeit zu den aufgeführten Schichten des TCP/IP Modells wichtige Befehle für die Kommunikation und Analyse innerhalb eines Netzwerks aufgeführt.

Adressen innerhalb der einzelnen Schichten

In den einzelnen Schichten gibt es die unterschiedlichsten Adressen bzw. auch Informationen, die für die Kommunikation wichtig sind. Diese Adressen sind in den jeweiligen Schichten unterschiedlich und haben verschiedene Bedeutungen.

Network Access Layer

- **MAC-Adressen:** Eindeutige Adressen von Hardware, die Geräten in einem lokalen Netzwerk zugewiesen sind.
- **Broadcast-Adressen:** Adressen, die verwendet werden, um Daten an alle Geräte in einem Netzwerk zu senden.
- ...

Internet Layer

- **IP-Adressen:** Eindeutige Adressen, die Geräten in einem IP-Netzwerk zugewiesen sind.
- **Subnetzmasken:** Informationen, die verwendet werden, um IP-Adressen in Netzwerke und Subnetze zu unterteilen.
- ...

Transport Layer

- **Portnummern:** Eindeutige Nummern, die Anwendungen auf einem Gerät identifizieren und den Datenverkehr zwischen ihnen steuern.
- **Sequenznummern:** Informationen, die verwendet werden, um die Reihenfolge von Datenpaketen zu verfolgen und sicherzustellen, dass sie korrekt empfangen werden.
- ...

Application Layer

- **URLs:** Eindeutige Adressen, die Ressourcen im Internet identifizieren.
- **Domain-Namen:** Menschlich lesbare Adressen, die IP-Adressen zugeordnet sind und die Navigation im Internet zu erleichtern.
- ...

Befehle zum Network Access Layer

ip l

In seiner ausgeschriebenen Form steht `ip l` für `ip link`. Dieser Befehl listet alle Netzinterfaces auf, die auf dem System verfügbar sind. Er zeigt Informationen wie den Status der Schnittstellen (`up / down`), die MAC-Adresse und andere relevante Details. Eine beispielhafte Ausgabe könnte folgendermaßen aussehen:

```

1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp3s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP mode DEFAULT group default
    qlen 1000
    link/ether f4:4d:30:6d:a5:64 brd ff:ff:ff:ff:ff:ff
3: wlp2s0: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN mode DEFAULT group default qlen 1000
    link/ether 10:f0:05:0e:d4:5e brd ff:ff:ff:ff:ff:ff
4: tun0: <BROADCAST,MULTICAST,NOARP,UP,LOWER_UP> mtu 1398 qdisc pfifo_fast state UNKNOWN mode DEFAULT group
    default qlen 1000
    link/ether ee:74:4c:d5:9a:2c brd ff:ff:ff:ff:ff:ff

```

Es gibt hierbei die unterschiedlichsten Arten von Netzkarten, die angezeigt bzw. aufgeführt werden können:

- **lo** : Loopback Interface, das für die interne Kommunikation des Systems verwendet wird. Entspricht immer dem Localhost, also der eigenen Maschine (127.0.0.1).
- **en** : Ethernet Interface, das für kabelgebundene Netzwerke benutzt wird.
- **wl** : Wireless LAN Interface, das für drahtlose Netzwerke eingesetzt wird.
- **tun** : Tunneling Interface, das für virtuelle private Netzwerke (VPNs) oder andere Tunnelverbindungen verwendet wird.

Hinter der Art des Interfaces folgt im Anschluss eine detaillierte Beschreibung des Platzes, an dem sich das Interface befindet:

- **p** : PCI (Peripheral Component Interconnect) Bus - Hardwareschnittstelle mit der die Netzwerkkarte mit dem Motherboard und der CPU kommuniziert.
- **s** : Slot - Spezifische Position auf dem PCI Bus. Hier ist folglich der Ort, wo die physische Hardware eingesteckt ist.

Beispiel: **enp3s0** bedeutet, dass es sich um ein Ethernet Interface handelt, das am PCI Bus 3 im Slot 0 angeschlossen ist.

IMPORTANT | Jeder Bus kann mehrer Slots besitzen!

ip -s l

Dieser Befehl ist eine erweiterte Version von **ip l** und zeigt zusätzlich Statistiken für jedes Interface an. Diese Statistiken umfassen die Anzahl der empfangenen und gesendeten Pakete, Fehler und Kollisionen. Die Ausgabe könnte wie folgt aussehen:

```

RX: bytes  packets  errors  dropped overrun mcast
    918788674 26603554 0        46533    0        6572075
TX: bytes  packets  errors  dropped carrier collsns
    34238378226 15183457 0         0         0         0

```

Man unterscheidet hier grundlegend zwischen:

- **RX** : Empfangene Daten.
- **TX** : Gesendete Daten.

Die folgenden Spalten haben die nachfolgende Bedeutung:

- **bytes** : Anzahl der empfangenen / gesendeten Bytes.
- **packets** : Anzahl der empfangenen / gesendeten Pakete.

- **errors** : Anzahl der Empfangs- und Sendefehler.
- **dropped** : Anzahl der verworfenen Pakete.
- **overrun** : Anzahl der Überläufe (wenn der Puffer voll ist). Das passiert immer dann, wenn Pakete schneller empfangen werden, als das System diese verarbeiten kann.
- **mcast** : Anzahl der empfangenen Multicast-Pakete. Zur Erläuterung:
 - **Unicast** : 1 Sender = 1 Empfänger (normale IP-Kommunikation).
 - **Broadcast** : 1 Sender = Alle im Netzwerk (255.255.255.255).
 - **Multicast** : 1 Sender = Gruppe von Empfängern.

arp

arp ist die Abkürzung für "Address Resolution Protocol". Dieser Befehl wird verwendet, um die Zuordnung von IP-Adressen zu MAC-Adressen im lokalen Netzwerk anzuzeigen. Die Ausgabe zeigt die ARP-Tabelle, die die IP-Adressen und die entsprechenden MAC-Adressen der Geräte im Netzwerk enthält. Das Problem, welches hierbei folglich gelöst wird, ist: "Ich kenne zwar die IP Adresse eines Gerätes, aber nicht die MAC-Adresse. Wie kann ich diese herausfinden?" Als Lösung wird ein ARP-Request als Broadcast an alle Geräte im Netzwerk gesendet. Das Gerät, welches die IP-Adresse besitzt, antwortet mit seiner MAC-Adresse. **arp** ist mehr oder weniger die Verbindung zwischen dem Network Access Layer und dem Internet Layer!

Sollte **arp** auf einem System noch nicht vorhanden sein, so muss dieses zunächst installiert werden:

```
sudo apt install net-tools
```

TERMINAL

Um sich im Anschluss die ARP-Tabelle anzeigen zu lassen, kann der Befehl **arp** verwendet werden:

```
arp
```

TERMINAL

Die Ausgabe könnte wie folgt aussehen:

Address	HWtype	HWaddress	Flags	Mask	Iface
router.local	ether	00:11:22:33:44:55	C		enp3s0
laptop.local	ether	00:11:22:33:44:56	C		enp3s0
printer.local	ether	00:11:22:33:44:57	C		enp3s0

TERMINAL

Die Spalten in der Ausgabe haben folgende Bedeutung:

- **Address** : IP-Adresse bzw. Hostname des Geräts.
- **HWtype** : Typ der Hardware (in der Regel **ether** für Ethernet).
- **HWaddress** : MAC-Adresse des Geräts.
- **Flags** : Status der Einträge (z.B. **C** für "Complete", was bedeutet, dass die Zuordnung vollständig ist).
- **Mask** : Netzmaske (in der Regel nicht relevant für ARP).
- **Iface** : Schnittstelle, über die die Zuordnung erfolgt ist (z.B. **enp3s0** für Ethernet).

Man kann den Befehl auch mit der Option **-a** (= **all**), wodurch eine mehr leserliche Ausgabe entsteht:

```
router.local (192.168.1.1) at 00:11:22:33:44:55 [ether] on enp3s0
laptop.local (192.168.1.2) at 00:11:22:33:44:56 [ether] on enp3s0
printer.local (192.168.1.3) at 00:11:22:33:44:57 [ether] on enp3s0
```

ip neigh

Der Befehl `ip neigh` bzw. `ip n` ist eine Alternative zu `arp` und sollte genutzt werden, da `arp` mittlerweile deprecated ist. Wesentlicher Unterschied zu `arp` und `ip n` ist, dass `ip n` auch IPv6 supported! Das obige Beispiel kann mit `ip n` wie folgt ausgegeben werden:

```
router.local dev enp3s0 lladdr 00:11:22:33:44:55 REACHABLE
laptop.local dev enp3s0 lladdr 00:11:22:33:44:56 REACHABLE
printer.local dev enp3s0 lladdr 00:11:22:33:44:57 REACHABLE
```

Die Ausgabe von `ip neigh` zeigt die Nachbarschaftsinformationen für die Netzwerkschnittstellen an. Die Spalten haben folgende Bedeutung:

- `router.local` : Hostname des Geräts.
- `dev enp3s0` : Die Schnittstelle, über die das Gerät erreichbar ist.
- `lladdr 00:11:22:33:44:55` : Die MAC-Adresse des Geräts. `lladdr` steht hierbei für `Link Layer Address`.
- `REACHABLE` : Der Status der Verbindung. Weitere mögliche Werte hierbei sind:
 - `STALE` : Das Gerät war erreichbar, aber die Informationen sind möglicherweise veraltet.
 - `DELAY` : Überprüfung der Erreichbarkeit läuft, Wartezeit vor neuer ARP-Anfrage.
 - `PROBE` : Das System führt aktiv eine ARP-Überprüfung durch.
 - `INCOMPLETE` : ARP-Anfrage läuft noch, keine MAC-Adresse verfügbar.
 - `FAILED` : Das Gerät ist nicht erreichbar oder antwortet nicht.
 - `PERMANENT` : Statisch konfigurierter Eintrag (manuell gesetzt).
 - `NOARP` : Das Interface unterstützt kein ARP (z.B. Point-to-Point-Verbindungen).

Befehle zum Internet Layer

Grundlegende Vorbereitung

Damit die nachfolgenden Befehle erfolgreich ausgeführt werden können, wird ein kleines Homelab mit zwei virtuellen Maschinen (VMs) benötigt. Die erste VM wurde bereits durch das Teilkapitel "Virtualisierung mit VBox" eingerichtet. Die VM muss allerdings noch angepasst bzw. abgeändert werden:

Schritte für die bestehende VM

1. Namen der VM auf `<User>_debian_client` setzen.
2. Netzwerkkarte von `Netzwerkbrücke` auf `Internes Netzwerk` mit dem Namen `intnet` setzen.

Schritte für die neue VM

1. Anpassen des Scripts für eine neue Maschine mit dem Namen `<User>_debian_server`.
2. Skript ausführen und die neue VM erstellen lassen.
3. Neben der Netzwerkkarte, die als Netzwerkbrücke konfiguriert ist, eine zweite Netzwerkkarte hinzufügen, die auf `Internes Netzwerk` mit dem Namen `intnet` gesetzt ist.

ip a

Der Befehl `ip a` (oder `ip addr`) wird verwendet, um die Netzwerkschnittstellen und deren IP-Adressen auf einem Linux-System anzuzeigen. Führt man diesen Befehl zum Beispiel auf der Maschine `<User>_debian_server` aus, so erhält man eine Ausgabe, die in etwa so aussieht:

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:44:ba:e6 brd ff:ff:ff:ff:ff:ff
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:0e:84:50 brd ff:ff:ff:ff:ff:ff
    inet6 fe80::a00:27ff:fe0e:8450/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

TERMINAL

Mit dem Befehl werden alle verfügbaren Netzwerkschnittstellen aufgelistet, einschließlich der Loopback-Schnittstelle (`lo`) und der physischen Netzwerkschnittstellen (`enp0s3`, `enp0s8`). Jede Schnittstelle zeigt ihre Konfiguration, einschließlich der IP-Adressen, des Status und anderer relevanter Informationen an.

Man kann allerdings auch gezielt nach einer Schnittstelle direkt suchen. Hierfür kann der Parameter `s` (= `show`) verwendet werden:

```
ip a s enp0s3
```

TERMINAL

Durch diesen Befehl wird die Ausgabe eingeschränkt:

```
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:44:ba:e6 brd ff:ff:ff:ff:ff:ff
```

TERMINAL

Auffällig ist, dass das Interface keine IP-Adresse zugewiesen hat. Um dieses Problem zu umgehen, muss dem Interface gesagt werden, dass es automatisch eine IP-Adresse beziehen soll.

Konfiguration der Schnittstelle für die Netzwerkbrücke via `/etc/network/interfaces`

Um eine Netzwerkschnittstelle, die als Netzwerkbrücke fungieren sollte, so zu konfigurieren, dass sie automatisch eine IP-Adresse bezieht, kann die Datei `/etc/network/interfaces` angepasst werden. Diese Datei enthält die Konfiguration für die Netzwerkschnittstellen des Systems. Die Konfiguration für die Schnittstelle `enp0s3` könnte wie folgt aussehen:

```
...
auto enp0s3
iface enp0s3 inet dhcp
...
```

TERMINAL

Hierbei bedeutet `auto enp0s3`, dass die Schnittstelle beim Systemstart automatisch aktiviert wird. Die Zeile `iface enp0s3 inet dhcp` gibt an, dass die Schnittstelle `enp0s3` eine IPv4-Adresse über DHCP beziehen soll. Möchte man eine IPv6-Adresse beziehen, so kann `inet dhcp` durch `inet6 dhcp` ersetzt werden.

Nach einem Systemneustart bzw. einem Neustart des Netzwerkdienstes sollte die Schnittstelle `enp0s3` automatisch eine IP-Adresse beziehen und auch die Ausgabe von `ip a` sollte entsprechend aktualisiert werden.

Vergabe einer IP-Adresse für das interne Netzwerk

Neben der Datei `/etc/network/interfaces` kann auch direkt der `ip` Befehl verwendet werden, um einer Schnittstelle eine IP-Adresse zuzuweisen. Die Syntax dafür lautet:

```
ip a(ddr) a(dd) <IP-Adresse>/<Subnetzmaske> dev <Schnittstelle>
```

TERMINAL

In diesem Fall könnte der Befehl für die Schnittstelle `enp0s8` wie folgt aussehen:

```
ip a a 10.0.147.1/24 dev enp0s8
```

TERMINAL

Da es natürlich immer wieder vorkommen kann, dass man bei der Konfiguration einen Fehler macht, kann man die Einstellung jederzeit wieder entfernen. Die Syntax dafür lautet:

```
ip a(ddr) d(elete) <IP-Adresse>/<Subnetzmaske> dev <Schnittstelle>
```

TERMINAL

Im Beispiel könnte der Befehl zum Entfernen der IP-Adresse von `enp0s8` wie folgt aussehen:

```
ip a d 10.0.147.1/24 dev enp0s8
```

TERMINAL

IMPORTANT

Der Befehl setzt eine IP-Adresse nur temporär für eine Netzwerksschnittstelle. Nach einem Neustart des Systems oder der Netzwerkschnittstelle geht diese Konfiguration verloren. Um die IP-Adresse dauerhaft zu setzen, sollte die Datei `/etc/network/interfaces` entsprechend angepasst werden.

Konfiguration einer statischen IP-Adresse für das internet Netzwerk

Um eine statische IP-Adresse für die Schnittstelle `enp0s8` zu konfigurieren, kann die Datei `/etc/network/interfaces` wie folgt angepasst werden:

```
...
auto enp0s8
iface enp0s8 inet static
    address 10.0.147.1/24
...
```

TERMINAL

Anpassung des Clients

Der Client im Homelab befindet sich rein im entsprechenden internen Netzwerk. Daher muss die Schnittstelle `enp0s3` so konfiguriert werden, dass sie eine IP-Adresse im gleichen Subnetz wie der Server erhält. Die Konfiguration könnte wie folgt aussehen:

```
ip a a 10.0.147.2/24 dev enp0s3
```

TERMINAL

Bereits mit dieser Einstellung können sich die beiden Maschinen gegenseitig erreichen.

ip r

Der Befehl `ip r` (oder `ip route`) wird verwendet, um die Routing-Tabelle eines Linux-Systems anzuzeigen. Diese Tabelle enthält Informationen darüber, wie Pakete an verschiedene Netzwerke weitergeleitet werden sollen. Wenn ein Paket an ein Ziel gesendet wird, das sich nicht im lokalen Netzwerk befindet, verwendet das System die Routing-

Tabelle, um zu bestimmen, an welches Gateway das Paket gesendet werden soll.

Die Ausgabe von `ip r` auf der Maschine `<User>_debian_client` könnte beispielsweise wie folgt aussehen:

```
10.0.147.0/24 dev enp0s3 proto kernel scope link src 10.0.147.2
169.254.0.0/16 dev enp0s3 scope link metric 1000
```

TERMINAL

In diesem Beispiel zeigt die erste Zeile, dass das Netzwerk `10.0.147.0/24` über die Schnittstelle `enp0s3` erreichbar ist und die Quell-IP-Adresse `10.0.147.2` verwendet wird. Die zweite Zeile zeigt ein Link-Local-Netzwerk an, das ebenfalls über die Schnittstelle `enp0s3` erreichbar ist.

Man kann auch gezielt neue Routen hinzufügen bzw. bestehende Routen entfernen. Die Syntax hierfür sieht folgendermaßen aus:

```
ip r a(dd) <Zielnetzwerk> via <Gateway-IP> dev <Schnittstelle>
ip r d(el) <Zielnetzwerk> via <Gateway-IP> dev <Schnittstelle>
```

TERMINAL

Möchte man zum Beispiel eine Standardroute zu einem Netzwerk setzen, so könnte der Befehl wie folgt aussehen:

```
ip r a default via 10.0.147.1 dev enp0s3
```

TERMINAL

Dieser Befehl fügt eine Standardroute hinzu, die alle Pakete, die nicht für ein bekanntes Netzwerk bestimmt sind, über das Gateway `10.0.147.1` und die Schnittstelle `enp0s3` leitet.

NOTE

Anstatt dem Schlüsselwort `default` kann auch das Zielnetzwerk `0/0` verwendet werden!

Abschließende Konfigurationsschritte

Ähnlich wie schon bei der Konfiguration für den Server, sind auch Routen und IPs, welche für den Client gesetzt werden, nur temporär. Folglich muss auch hier die `/etc/network/interfaces` Datei wieder entsprechend angepasst werden:

```
...
auto enp0s3
iface enp0s3 inet static
    address 10.0.147.2/24
    gateway 10.0.147.1
...
```

TERMINAL

Durch diese Konfiguration werden dauerhaft die Einträge für die IP, welcher auch via `ip a a`, sowie der Eintrag für die Standardroute, der auch mit Hilfe von `ip r a`, gesetzt.

Auf dem Router selbst muss außerdem das Forwarding, also die Weiterleitung von Paketen zwischen den Netzwerken, aktiviert werden. Dies kann in der Regel durch Setzen des Parameters `net.ipv4.ip_forward` auf `1` in der Datei `/etc/sysctl.conf` erfolgen. Die Einstellung bewirkt dabei, dass der Router Pakete, die an ihn gerichtet sind, auch an andere Netzwerke weiterleitet. Sollte die Datei nicht vorhanden sein und z. B. Debian vollständig auf `/etc/sysctl.d/` ausgelagert ist, so kann die Einstellung auch in einer neuen Datei, z. B. `/etc/sysctl.d/99-ipforward.conf` vorgenommen werden. Der entsprechende Eintrag könnte wie folgt aussehen:

```
net.ipv4.ip_forward=1
```

TERMINAL

Nach einer Änderung der Datei muss der Befehl `sysctl -p` ausgeführt werden, damit die Änderungen übernommen werden:

```
sysctl -p /etc/sysctl.d/99-ipforward.conf
```

TERMINAL

Weiterhin muss auf dem Router NAT konfiguriert werden. Hierfür wird der `iptables` Befehl verwendet, um die Regeln für die Network Address Translation (NAT) festzulegen. Das entsprechende Paket, was hierfür installiert werden muss ist `iptables`:

```
apt install iptables
```

TERMINAL

Im Anschluss können die nachfolgenden Regeln hinzugefügt werden:

```
iptables -t nat -A POSTROUTING -o enp0s3 -j MASQUERADE
iptables -A FORWARD -i enp0s8 -o enp0s3 -j ACCEPT
iptables -A FORWARD -i enp0s3 -o enp0s8 -m state --state RELATED,ESTABLISHED -j ACCEPT
```

TERMINAL

Exkurs zu iptables

- `-t` : `t` steht für `table`. Über den Parameter wird die Tabelle angegeben, auf die die Regel angewendet wird. In diesem Fall ist es die `nat` Tabelle, die für Network Address Translation verwendet wird.
- `-A` : `A` steht für `append`. Die Regel wird folglich am Ende einer Kette hinzugefügt. Die entsprechende Kette kann ein Wert wie `INPUT`, `OUTPUT`, `FORWARD`, `PREROUTING` oder `POSTROUTING` sein:
 - `INPUT` : Regeln für eingehende Pakete.
 - `OUTPUT` : Regeln für ausgehende Pakete.
 - `FORWARD` : Regeln für Pakete, die weitergeleitet werden.
 - `PREROUTING` : Regeln, die angewendet werden, bevor die Pakete geroutet werden.
 - `POSTROUTING` : Regeln, die angewendet werden, nachdem die Pakete geroutet wurden.
- `-o` : `o` ist die Abkürzung für `out-interface`, also das ausgehende Interface, auf das eine Regel angewendet wird.
- `-j` : `j` steht für `jump`. Mit diesem Parameter wird angegeben, was mit den Paketen geschehen soll, welche die Regel erfüllen. Mögliche Werte in diesem Zusammenhang sind:
 - `ACCEPT` : Die Pakete werden akzeptiert und weitergeleitet.
 - `DROP` : Die Pakete werden verworfen und nicht weitergeleitet.
 - `REJECT` : Die Pakete werden verworfen, aber der Absender erhält eine Benachrichtigung.
 - `MASQUERADE` : Diese Aktion wird verwendet, um die Quell-IP-Adresse der Pakete zu maskieren, sodass sie die IP-Adresse des Routers verwenden, wenn sie das interne Netzwerk verlassen. Dies ist besonders nützlich für NAT.
 - `SNAT` / `DNAT` : Diese Aktionen werden verwendet, um die Quell- (SNAT) oder Ziel-IP-Adresse (DNAT) der Pakete zu ändern. Dies ist nützlich, um den Datenverkehr zwischen verschiedenen Netzwerken zu steuern.
 - `LOG` : Protokollierung von Paketen.
- `-i` : `i` ist die Abkürzung für `in-interface`, also das Interface, über das ein Paket empfangen wird.
- `-m` : `m` ist der Parameter für `match`, also Übereinstimmungen. Der Parameter wird verwendet, um zusätzliche Bedingungen für die Regel festzulegen. Über diesen Parameter können verschiedene Module wie z. b. `state` geladen werden. Das Modul `state` ermöglicht die Verfolgung von Verbindungszuständen und wird häufig in

Firewall-Regeln verwendet, um den Status von Verbindungen zu überprüfen. Mit dem Parameter `--state` können spezifische Zustände wie `NEW`, `ESTABLISHED`, `RELATED` oder `INVALID` angegeben werden:

- `NEW` : Ein neues Paket, das eine neue Verbindung initiiert.
- `ESTABLISHED` : Ein Paket, das zu einer bereits bestehenden Verbindung gehört.
- `RELATED` : Ein Paket, das zu einer bestehenden Verbindung gehört, aber nicht direkt Teil dieser Verbindung ist (z. B. eine Antwort auf eine vorherige Anfrage).
- `INVALID` : Ein Paket, das als ungültig betrachtet wird und nicht weiterverarbeitet werden sollte.

Erläuterung zu den einzelnen Regeln:

- `iptables -t nat -A POSTROUTING -o enp0s3 -j MASQUERADE` : Diese Regel maskiert die Quell-IP-Adresse der Pakete, die über die Schnittstelle `enp0s3` gesendet werden. Dadurch wird die IP-Adresse des Routers als Absenderadresse verwendet, wenn Pakete das interne Netzwerk verlassen.
- `iptables -A FORWARD -i enp0s8 -o enp0s3 -j ACCEPT` : Diese Regel erlaubt den Datenverkehr, der von der Schnittstelle `enp0s8` (dem internen Netzwerk) zur Schnittstelle `enp0s3` (dem externen Netzwerk) weitergeleitet wird.
- `iptables -A FORWARD -i enp0s3 -o enp0s8 -m state --state RELATED,ESTABLISHED -j ACCEPT` : Diese Regel erlaubt den Rückverkehr von bereits etablierten Verbindungen, die von der Schnittstelle `enp0s3` (dem externen Netzwerk) zur Schnittstelle `enp0s8` (dem internen Netzwerk) zurückkehren.

Um letztendlich vom Client aus Anfragen ins Internet zu senden, muss auf dem Client noch ein funktionierender Nameserver konfiguriert werden. Dies kann in der Regel durch Setzen des Parameters `nameserver` in der Datei `/etc/resolv.conf` erfolgen. Ein Beispiel für die Konfiguration könnte wie folgt aussehen:

```
nameserver 8.8.8.8
```

TERMINAL

Dieser Eintrag weist das System an, den DNS-Server von Google zu verwenden, um Domainnamen in IP-Adressen aufzulösen. Alternativ kann auch der Nameserver des eigenen Internetanbieters oder ein anderer öffentlicher DNS-Server verwendet werden.

Testen der Verbindungen zu verschiedenen Hosts

Um Verbindungen zu überprüfen, gibt es unterschiedlichste Mechanismen:

ping

Mit dem Befehl `ping` kann überprüft werden, ob ein bestimmter Host im Netzwerk erreichbar ist. Der Befehl sendet ICMP Anfragen an die Zieladresse und wartet auf eine Antwort. Ein erfolgreicher `ping` Befehl zeigt an, dass die Netzwerkverbindung funktioniert.

NOTE

ICMP ist die Abkürzung für Internet Control Message Protocol . Es ist ein Netzwerkprotokoll, das verwendet wird, um Fehler- und Statusmeldungen im Internet zu übertragen. ICMP wird häufig für Diagnosetools wie `ping` und `traceroute` verwendet.

Der typische Aufbau für den `ping` Befehl sieht wie folgt aus:

```
ping 10.0.147.1
```

TERMINAL

Das Ergebnis hat immer einen ähnlichen Aufbau:

```
PING 10.0.147.1 (10.0.147.1) 56(84) bytes of data.
64 bytes from 10.0.147.1: icmp_seq=1 ttl=64 time=1.44 ms
64 bytes from 10.0.147.1: icmp_seq=2 ttl=64 time=0.741 ms
64 bytes from 10.0.147.1: icmp_seq=3 ttl=64 time=0.485 ms
64 bytes from 10.0.147.1: icmp_seq=4 ttl=64 time=0.548 ms
^C
--- 10.0.147.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3104ms
rtt min/avg/max/mdev = 0.485/0.803/1.441/0.379 ms
```

Erläuterung der aufgeführten Ausgabe:

- **56(84)** : Die Größe der gesendeten Pakete in Bytes. In diesem Fall sind es 56 Bytes, aber aufgrund von Header-Informationen beträgt die Gesamtgröße 84 Bytes.
- **64 bytes...** : Die Antwort des Zielhosts, die die Größe der empfangenen Pakete angibt.
- **icmp_seq=1** : Die Sequenznummer der ICMP-Anfrage. Diese Nummer wird erhöht, wenn weitere Anfragen gesendet werden.
- **ttl=64** : Die Time-to-Live (TTL) des Pakets. Dies gibt an, wie viele Hops (Router) das Paket durchlaufen darf, bevor es verworfen wird. Ein Wert von 64 ist häufig der Standardwert.
- **time=1.44 ms** : Die Zeit, die benötigt wurde, um die Antwort zu erhalten, gemessen in Millisekunden (ms). Dies gibt an, wie schnell die Verbindung ist.

Für den `ping` Befehl gibt es noch weitere interessante Parameter:

- **-c <Anzahl>** : Begrenzt die Anzahl der gesendeten Pakete. Zum Beispiel `ping -c 4` führt dazu, dass lediglich vier Pakete gesendet werden.
- **-i <Intervall>** : Legt das Intervall zwischen den gesendeten Paketen fest. Zum Beispiel `ping -i 2` sendet alle zwei Sekunden ein Paket.
- **-s <Paketgröße>** : Legt die Größe der gesendeten Pakete fest. Zum Beispiel `ping -s 128` sendet Pakete mit einer Größe von 128 Bytes.

fping

`fping` ist mehr oder weniger eine Erweiterung zum normalen `ping` Befehl. Der Hauptunterschied besteht darin, dass `fping` mehrere Hosts gleichzeitig anpingen kann, was es zu einem nützlichen Werkzeug für die Überwachung von Netzwerken macht. Es existieren dabei die nachfolgenden Möglichkeiten:

- Paralleles anpingen mehrerer Hosts: `fping host1 host2 host3`.
- Netzwerk-Scanning: Hierrüber können alle Hosts in einem bestimmten Netzwerk angepingt werden. Es existieren dafür die nachfolgenden Parameter:
 - **-g** : Gibt einen IP-Adressbereich an, der gescannt werden soll. Zum Beispiel `fping -g 10.0.147.0/24` pingt alle Hosts im Subnetz 10.0.147.0 an.
 - **-a** : Zeigt nur die Hosts an, die erreichbar sind. Zum Beispiel `fping -a -g 10.0.147.0/24 2>/dev/null`.

NOTE

`fping` ist nicht standardmäßig auf allen Distributionen installiert. Es muss möglicherweise über den Paketmanager installiert werden, z. B. mit `apt install fping`.

nmap

nmap (Network Mapper) ist ein leistungsstarkes Tool zur Netzwerkerkennung und Sicherheitsüberprüfung. Es wird häufig verwendet, um Hosts und Dienste in einem Netzwerk zu identifizieren, Sicherheitslücken zu erkennen und Netzwerktopologien zu analysieren. Ähnlich zum Befehl **ping** kann hier ebenfalls ein Netzwerk gescannt werden. Der Befehl sieht wie folgt aus:

```
nmap -sn 10.0.147.0/24
```

TERMINAL

Die Ausgabe ist allerdings etwas umfangreicher als die des **ping** Befehls:

```
Starting Nmap 7.93 ( https://nmap.org ) at 2025-08-19 19:50 CEST
Nmap scan report for 10.0.147.1
Host is up (0.0012s latency).
MAC Address: 08:00:27:0E:84:50 (Oracle VirtualBox virtual NIC)
Nmap scan report for 10.0.147.2
Host is up.
Nmap done: 256 IP addresses (2 hosts up) scanned in 35.03 seconds
```

TERMINAL

Testen von Routen

traceroute

traceroute ist ein Tool, das verwendet wird, um den Pfad zu verfolgen, den Pakete von einem Host zu einem anderen Host im Netzwerk nehmen. Es zeigt die Zwischenstationen (Router) an, die die Pakete durchlaufen, und hilft dabei, Netzwerkprobleme zu diagnostizieren. Eine beispielhafte Verwendung sieht folgendermaßen aus:

```
traceroute www.edvschule-plattling.de
```

TERMINAL

Als Ergebnis erhält man eine ähnliche Ausgabe wie die folgende:

```

traceroute to edvschule-plattling.de (212.77.167.110), 30 hops max, 60 byte packets
 1 _gateway (10.0.147.1) 0.325 ms 0.417 ms 0.327 ms
 2 192.168.178.1 (192.168.178.1) 9.471 ms 9.218 ms 8.941 ms
 3 p3e9bf2bc.dip0.t-ipconnect.de (62.155.242.188) 58.848 ms 58.787 ms 58.654 ms
 4 f-ed13-i.F.DE.NET.DTAG.DE (217.5.118.238) 58.752 ms f-ed13-i.F.DE.NET.DTAG.DE (217.5.118.230) 58.615 ms
217.5.118.226 (217.5.118.226) 58.501 ms
 5 80.150.169.79 (80.150.169.79) 58.107 ms 58.001 ms 57.786 ms
 6 r2-fra1-de.as5405.net (94.103.180.5) 57.620 ms 26.917 ms 26.720 ms
 7 45.153.83.223 (45.153.83.223) 25.896 ms 31.873 ms 31.799 ms
 8 cr1.rz1.r-kom.net (185.39.20.106) 31.660 ms 31.603 ms 31.544 ms
 9 * * *
10 * * *
11 * * *
12 * * *
13 * * *
14 * * *
15 * * *
16 * * *
17 * * *
18 * * *
19 * * *
20 * * *
21 * * *
22 * * *
23 * * *
24 * * *
25 * * *
26 * * *
27 * * *
28 * * *
29 * * *
30 * * *

```

Erläuterung der Ausgabe:

- `traceroute to edvschule-plattling.de (212.77.167.110), 30 hops max, 60 byte packets` : Dies zeigt das Ziel der Traceroute an, einschließlich der IP-Adresse des Ziels und der maximalen Anzahl von Hops (Zwischenstationen), die verfolgt werden sollen.
- Die nachfolgenden Zeilen zeigen die einzelnen Hops an, die die Pakete durchlaufen, einschließlich der IP-Adressen und der Latenzzeiten (in Millisekunden) für jeden Hop.

IMPORTANT

Die Ausgabe kann je nach Netzwerk und Routing unterschiedlich sein. Einige Hops können möglicherweise nicht antworten, was durch `* * *` angezeigt wird. Dies kann auf Firewall-Regeln oder andere Netzwerkprobleme hinweisen. Es bedeutet aber nicht zwangsläufig, dass das Ziel nicht erreichbar ist, sondern vielmehr, dass ein Großteil der Server / Router so konfiguriert sind, dass sie nicht auf `traceroute` Anfragen antworten!

`traceroute` kennt hierbei noch weitere Parameter, die für die Ausgabe interessant sein können:

- `-I` : Im Standard verwendet `traceroute` UDP. Mit dem Parameter `-I` wird stattdessen das ICMP Protokoll verwendet, was in vielen Netzwerken besser funktioniert.
- `-p <Port>` : Mit diesem Parameter kann der Port angegeben werden, der für die Anfrage verwendet werden soll. Zum Beispiel `-p 80` für den HTTP-Port.

Befehle zum Transport Layer

Ports?

Im Transport Layer spielen Ports eine wesentliche Rolle. Ports sind dabei eindeutige Nummern, die Anwendungen auf einem Gerät identifizieren und den Datenverkehr zwischen ihnen steuern. Sie ermöglichen es, mehrere Dienste auf demselben Gerät zu betreiben, indem sie jedem Dienst eine eigene Portnummer zuweisen. Zum Beispiel verwendet der Webserver in der Regel Port 80 für HTTP und Port 443 für HTTPS. Portnummern können Werte zwischen 0 und 65535 sein. Generell gesehen gibt es drei wesentliche Portbereiche:

- 0-1023 : Diese Ports sind als "Well-Known Ports" bekannt und werden von Systemdiensten und Protokollen wie HTTP (Port 80), HTTPS (Port 443) und FTP (Port 21) verwendet.
- 1024-49151 : Diese Ports sind als "Registered Ports" bekannt und können von Anwendungen verwendet werden, die nicht zu den Well-Known Ports gehören. Sie sind oft für spezifische Anwendungen reserviert, aber nicht so streng reguliert wie die Well-Known Ports.
- 49152-65535 : Diese Ports sind als "Dynamic" oder "Private Ports" bekannt und werden von Anwendungen dynamisch zugewiesen. Sie werden oft für temporäre Verbindungen verwendet, wenn eine Anwendung eine Verbindung zu einem Server herstellt.

netstat

`netstat` (Network Statistics) ist ein Tool, um Netzwerkverbindungen, Routing-Tabellen und eine Vielzahl von Netzwerkstatistiken anzuzeigen. Es kann verwendet werden, um Informationen über aktive Verbindungen, offene Ports und die Nutzung von Netzwerkressourcen zu erhalten. `netstat` ist besonders nützlich für die Fehlersuche und Überwachung von Netzwerkverbindungen. Wichtige Parameter im Zusammenhang mit `netstat` sind:

- `t` : Ermöglicht die Anzeige von TCP-Verbindungen.
- `u` : Ermöglicht die Anzeige von UDP-Verbindungen.
- `l` : Zeigt nur die Listening-Ports an.
- `p` : Zeigt den Prozess an, der die Verbindung hält.
- `e` : Zeigt erweiterte Statistiken an.
- `n` : Zeigt die Adressen und Ports numerisch an, anstatt sie aufzulösen.

Ein Beispiel für `netstat` könnte folgendermaßen aussehen:

```
netstat -teln
```

TERMINAL

```
Aktive Internetverbindungen (ohne Server)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       Benutzer   Inode
PID/Program name
tcp        0      0 10.0.147.2:41664        10.0.147.1:22          VERBUNDEN   1000       29816      3248/ssh
```

TERMINAL

Erläuterung zur Ausgabe:

- `Proto` : Protokolltyp (TCP/UDP). Im Beispiel wird TCP verwendet.
- `Recv-Q` und `Send-Q` : Anzahl der Bytes, die im Empfangs- bzw. Sendepuffer warten.
- `Local Address` : Lokale IP-Adresse und Port.
- `Foreign Address` : Remote-IP-Adresse und Port.

- State : Verbindungsstatus (z.B. VERBUNDEN , WARTEND).
- Benutzer : Benutzer, der die Verbindung hält.
- Inode : Nummer der Verbindung.
- PID/Program name : ID des Prozesses und Name des Programms, das die Verbindung hält.

Im Endeffekt besagt die Ausgabe, dass der Prozess mit der PID 3248 (in diesem Fall `ssh`) eine aktive TCP-Verbindung von der lokalen IP-Adresse 10.0.147.2 und dem Port 41664 zur Remote-IP-Adresse 10.0.147.1 und dem Port 22 hat.

Ergänzt man den Befehl noch um den `-l` Parameter, dann sieht die Ausgabe wie folgt aus:

```
netstat -teltn
```

TERMINAL

Die Ausgabe könnte wie folgt aussehen:

```

Aktive Internetverbindungen (Nur Server)
Proto Recv-Q Send-Q Local Address          Foreign Address         State       Benutzer   Inode
PID/Program name
tcp        0      0 127.0.0.1:631          0.0.0.0:*                LISTEN      0          15797
699/cupsd
tcp6       0      0 :::631                 :::*                      LISTEN      0          15796
699/cupsd

```

TERMINAL

Mit dem Parameter `-l` werden nur die Listening-Ports angezeigt, also die Ports, auf denen der Server auf eingehende Verbindungen wartet.

nmap

Der Befehl `nmap` (Network Mapper) wurde bereits innerhalb der Befehle zum Internet Layer kurz angesprochen.

`nmap` wird dabei nicht nur zum Scannen eines Netzwerks verwendet, sondern kann auch eingesetzt werden, damit innerhalb eines entfernten Rechners die offenen Ports angezeigt werden. Dies ist besonders nützlich, um zu überprüfen, welche Dienste auf einem Server verfügbar sind und ob diese sicher konfiguriert sind. `nmap` kann auch verwendet werden, um Sicherheitslücken zu identifizieren und potenzielle Angriffsvektoren zu erkennen:

```
nmap 10.0.147.1
```

TERMINAL

```

Starting Nmap 7.93 ( https://nmap.org ) at 2025-08-21 14:41 CEST
Nmap scan report for 10.0.147.1
Host is up (0.00032s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
MAC Address: 08:00:27:0E:84:50 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 16.79 seconds

```

TERMINAL

Im Beispiel sieht man, dass der Host generell erreichbar ist und dass der Port 22 (SSH) offen ist. Alle anderen Ports sind geschlossen. Die Ausgabe enthält auch die MAC-Adresse des Geräts, was nützlich sein kann, um den Gerätetyp zu identifizieren.

`nmap` bietet eine Vielzahl von Optionen, um die Scans anzupassen, z.B. um spezifische Ports zu scannen, die Scan-Geschwindigkeit zu ändern oder detailliertere Informationen über die Dienste zu erhalten, die auf den offenen Ports laufen. Einige nützliche Optionen sind:

- `-sS` : Führt einen SYN-Scan (Synchronize Scan) durch, um offene Ports zu identifizieren. Hierbei handelt es sich um den Standard bei `nmap` . Dieser Scan ist in der Regel sehr schnell und wird wenig protokolliert. Der Scan muss weiterhin als Root ausgeführt werden.
- `-sT` : Hierbei handelt es sich um einen TCP-Connect-Scan, der einen kompletten 3-Wege-Handshake durchführt. Im Vergleich zum Synchronize Scan wird dieser meist protokolliert und auch als Angriff gewertet. Das Prinzip des 3-Wege-Handshakes sieht folgendermaßen aus:
 - `SYN` : Der Client sendet ein SYN-Paket an den Server, um eine Verbindung zu initiieren.
 - `SYN-ACK` : Der Server antwortet mit einem SYN-ACK-Paket, um die Verbindung zu bestätigen.
 - `ACK` : Der Client sendet ein ACK-Paket, um die Verbindung zu bestätigen.
- `-sU` : Führt einen UDP-Scan durch, um offene UDP-Ports zu identifizieren. Dieser Scan ist in der Regel langsamer und weniger zuverlässig als der TCP-Scan, da UDP verbindungslos ist und keine Handshake-Mechanismen verwendet. Um eine Kombination aus UDP- und Stealth-Scan durchzuführen, kann der Parameter `-sSU` benutzt werden.
- `-p-` : Mit Hilfe dieser Option können alle Ports (1-65535) gescannt werden. Standardmäßig scannt `nmap` nur die 1000 am häufigsten verwendeten Ports. Mit dieser Option dauert `nmap` sehr lange.
- `-p80,443` : Scannt nur die angegebenen Ports (in diesem Fall Port 80 und 443).
- `-sV` : Versucht, die Version der Dienste zu ermitteln, die auf den offenen Ports laufen. Dies kann nützlich sein, um zu überprüfen, ob die Dienste auf dem neuesten Stand sind und keine bekannten Sicherheitslücken aufweisen.
- `-O` : Versucht, das Betriebssystem des Zielhosts zu erkennen. Dies kann nützlich sein, um zu verstehen, welche Sicherheitsmaßnahmen möglicherweise erforderlich sind.
- `-sn` : Hierrüber werden alle erreichbaren IPs innerhalb eines Netzwerks gescannt. Möchte man das ganze Ergebnis in eine Datei auslagern, so kann die Option `-oG <File>` verwendet werden. Das `oG` steht in diesem Zusammenhang für "Output in Greppable Format". Dies ist ein einfaches Textformat, das leicht von anderen Programmen verarbeitet werden kann.

Weitere Befehle und Toolings

`tcpdump`

Bei `tcpdump` handelt es sich um eine Tool innerhalb der Kommandozeile, womit der eingehende als auch ausgehende Netzwerkverkehr überwacht und analysiert werden kann. Für Netzwerkadministratoren ist es ein unverzichtbares Werkzeug, damit Netzwerkprobleme diagnostiziert und Sicherheitsvorfälle untersucht werden können. Unabhängig vom Namen kann nicht nur der TCP-Verkehr, sondern auch andere Protokolle wie ARP, UDP, ICMP und viele mehr überwacht werden. Um `tcpdump` verwenden zu können, muss es zunächst installiert werden:

```
sudo apt install tcpdump
```

TERMINAL

Führt man `tcpdump` im Anschluss aus, so werden alle entsprechenden Pakete aufgezeichnet, bis man den Prozess mit `Ctrl + C` beendet. Die Ausgabe kann dabei sehr umfangreich sein, weshalb es sinnvoll ist, Filter zu verwenden, um nur die relevanten Pakete anzuzeigen. Ein einfaches Beispiel für die Verwendung von `tcpdump` könnte wie folgt aussehen:


```
tcpdump -i enp3s0 port 80
```

Erläuterung der wichtigen Parameter:

- `-i enp3s0` : Gibt die Schnittstelle an, die überwacht werden soll (in diesem Fall `enp3s0`).
- `port 80` : Filtert die Pakete, um nur den Verkehr auf Port 80 (HTTP) anzuzeigen.
- `-n` : Verhindert die Auflösung von Hostnamen und zeigt stattdessen IP-Adressen an.
- `host <IP-Adresse>`: Filtert die Pakete für eine bestimmte IP-Adresse.
- `-v` : Erhöht die Detailgenauigkeit der Ausgabe.
- `-c 10` : Beendet die Überwachung nach 10 Paketen.
- `-w datei.pcap` : Speichert die Pakete in einer Datei im PCAP-Format, die später mit Wireshark oder einem ähnlichen Tool analysiert werden kann.
- `-l` : Zeigt die Ausgabe in Echtzeit an, ohne sie zu puffern. Dies ist nützlich, um die Pakete sofort zu sehen, während sie empfangen werden.

Beispiele für die Verwendung von `tcpdump` :

```
tcpdump -lni enp0s3
```

Im Beispiel wird der Netzwerkverkehr auf der Schnittstelle `enp0s3` in Echtzeit angezeigt. Dabei wird die Ausgabe nicht gepuffert, sodass die Pakete sofort sichtbar sind (= `-l`). Es werden weiterhin keine Hostnamen aufgelöst, sondern nur IP-Adressen angezeigt (= `-n`).

Eine mögliche Ausgabe des Befehls könnte folgendermaßen aussehen:

```
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enp0s3, link-type EN10MB (Ethernet), snapshot length 262144 bytes
17:03:43.262410 IP6 fe80::a00:27ff:fe0d:721b > ff02::2: ICMP6, router solicitation, length 16
17:07:01.721020 IP 10.0.147.2 > 10.0.147.1: ICMP echo request, id 4176, seq 1, length 64
17:07:01.721818 IP 10.0.147.1 > 10.0.147.2: ICMP echo reply, id 4176, seq 1, length 64
17:07:02.721982 IP 10.0.147.2 > 10.0.147.1: ICMP echo request, id 4176, seq 2, length 64
17:07:02.722753 IP 10.0.147.1 > 10.0.147.2: ICMP echo reply, id 4176, seq 2, length 64
17:07:03.724410 IP 10.0.147.2 > 10.0.147.1: ICMP echo request, id 4176, seq 3, length 64
17:07:03.725732 IP 10.0.147.1 > 10.0.147.2: ICMP echo reply, id 4176, seq 3, length 64
17:07:04.725547 IP 10.0.147.2 > 10.0.147.1: ICMP echo request, id 4176, seq 4, length 64
17:07:04.725882 IP 10.0.147.1 > 10.0.147.2: ICMP echo reply, id 4176, seq 4, length 64
17:07:05.758738 IP 10.0.147.2 > 10.0.147.1: ICMP echo request, id 4176, seq 5, length 64
17:07:05.759547 IP 10.0.147.1 > 10.0.147.2: ICMP echo reply, id 4176, seq 5, length 64
17:07:06.760741 IP 10.0.147.2 > 10.0.147.1: ICMP echo request, id 4176, seq 6, length 64
17:07:06.761971 IP 10.0.147.1 > 10.0.147.2: ICMP echo reply, id 4176, seq 6, length 64
17:07:06.781952 ARP, Request who-has 10.0.147.1 tell 10.0.147.2, length 28
17:07:06.783805 ARP, Reply 10.0.147.1 is-at 08:00:27:0e:84:50, length 46
17:07:06.904132 ARP, Request who-has 10.0.147.2 tell 10.0.147.1, length 46
17:07:06.904160 ARP, Reply 10.0.147.2 is-at 08:00:27:cd:72:1b, length 28
^C
17 packets captured
17 packets received by filter
0 packets dropped by kernel
```

```
tcpdump -lni enp0s3 arp
```

Im Beispiel wird nur der ARP-Verkehr auf der Schnittstelle `enp0s3` angezeigt. Dies ist nützlich, um die ARP-Anfragen und -Antworten im Netzwerk zu überwachen.

```
tcpdump -lni enp0s3 host 10.0.147.1
```

TERMINAL

Das Beispiel zeichnet den gesamten Verkehr zu und von der IP-Adresse `10.0.147.1` auf der Schnittstelle `enp0s3` auf.

```
tcpdump -lni enp0s3 host 10.0.147.1 and port 80
```

TERMINAL

Im Beispiel wird der HTTP-Verkehr (Port 80) zu und von der IP-Adresse `10.0.147.1` auf der Schnittstelle `enp0s3` aufgezeichnet.

```
tcpdump -s0 -w capture.pcap -lni enp0s3 host 10.0.147.1 -c 10
```

TERMINAL

Mit diesem Beispiel wird der Verkehr zu und von der IP-Adresse `10.0.147.1` auf der Schnittstelle `enp0s3` aufgezeichnet und in der Datei `capture.pcap` im PCAP-Format gespeichert. Die Überwachung wird nach 10 Paketen beendet. `-s0` stellt sicher, dass die gesamte Paketgröße erfasst wird, anstatt nur die ersten 68 Bytes (Standardgröße).

Wireshark

Wireshark ist ein grafisches Netzwerkprotokoll-Analyse-Tool, das eine detaillierte Analyse des Netzwerkverkehrs ermöglicht. Es ist besonders nützlich für die Fehlersuche und Sicherheitsüberwachung in Netzwerken. Mit Wireshark können Benutzer den Netzwerkverkehr in Echtzeit überwachen, Pakete aufzeichnen und analysieren sowie Protokolle und deren Interaktionen untersuchen. `tcpdump` kann dabei verwendet werden, um Pakete aufzuzeichnen, die dann in Wireshark importiert und analysiert werden können.

Um Wireshark zu installieren, kann auch wieder der Paketmanager verwendet werden:

```
apt install wireshark
```

TERMINAL

Damit auch der angemeldete User das Programm nutzen kann, muss dieser zur Gruppe `wireshark` hinzugefügt werden:

```
usermod -aG wireshark <User>
```

TERMINAL

Nach der Installation kann Wireshark über das Anwendungsmenü gestartet werden. Die Benutzeroberfläche von Wireshark ist intuitiv und ermöglicht es, den Netzwerkverkehr in Echtzeit zu überwachen, Filter anzuwenden und detaillierte Informationen zu jedem Paket anzuzeigen. Man unterscheidet hierbei mehrere Filter:

- Mitschnittfilter:

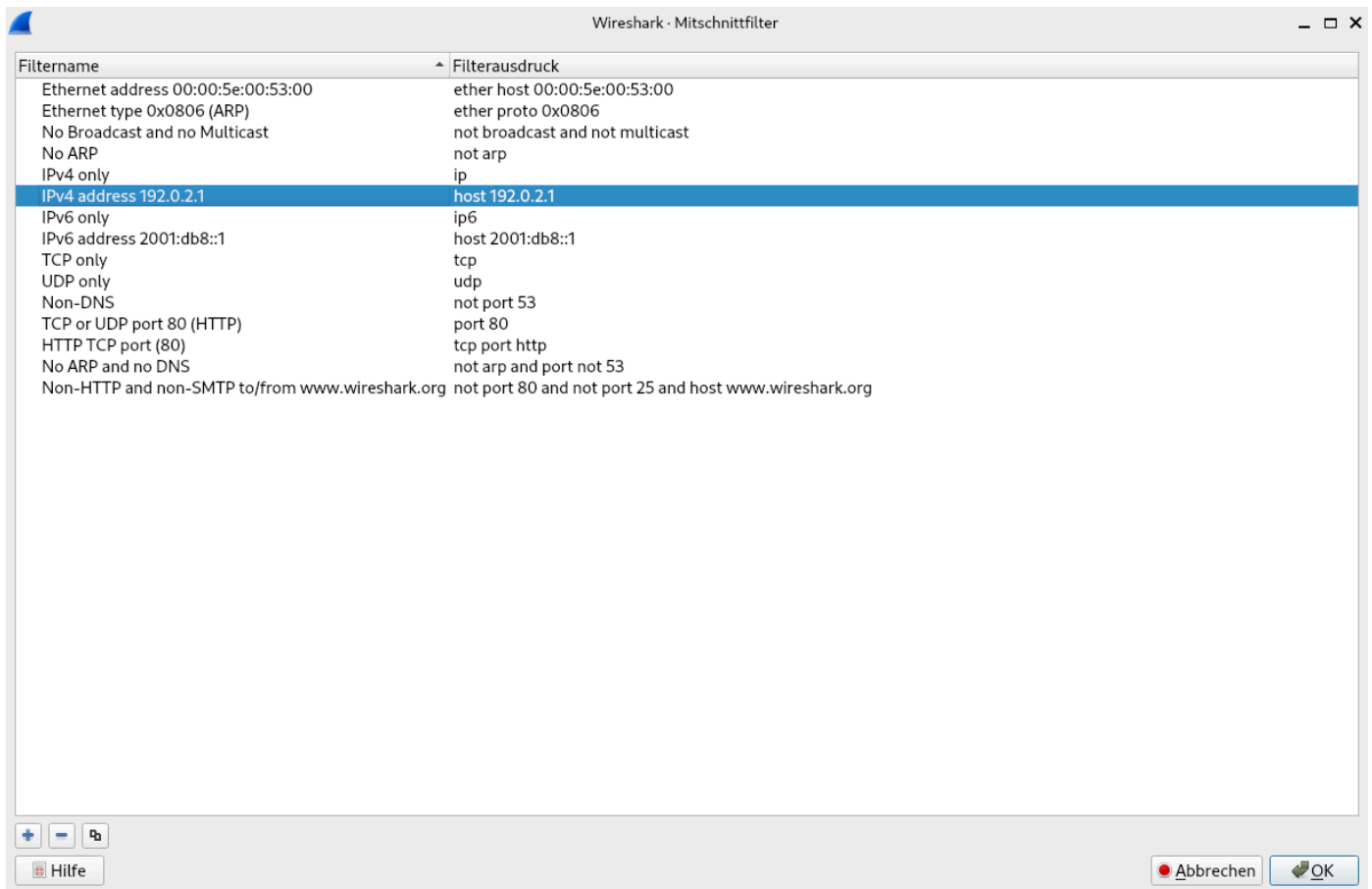


Abbildung 1. Mitschnittfilter

Hierbei handelt es sich um Filter, die angewendet werden, bevor der Netzwerkverkehr aufgezeichnet wird. Diese Filter bestimmen, welche Pakete erfasst und gespeichert werden. Beispiele für Mitschnittfilter sind zum Beispiel IP-Adressen, Protokolle oder Ports.

- Anzeigefilter:

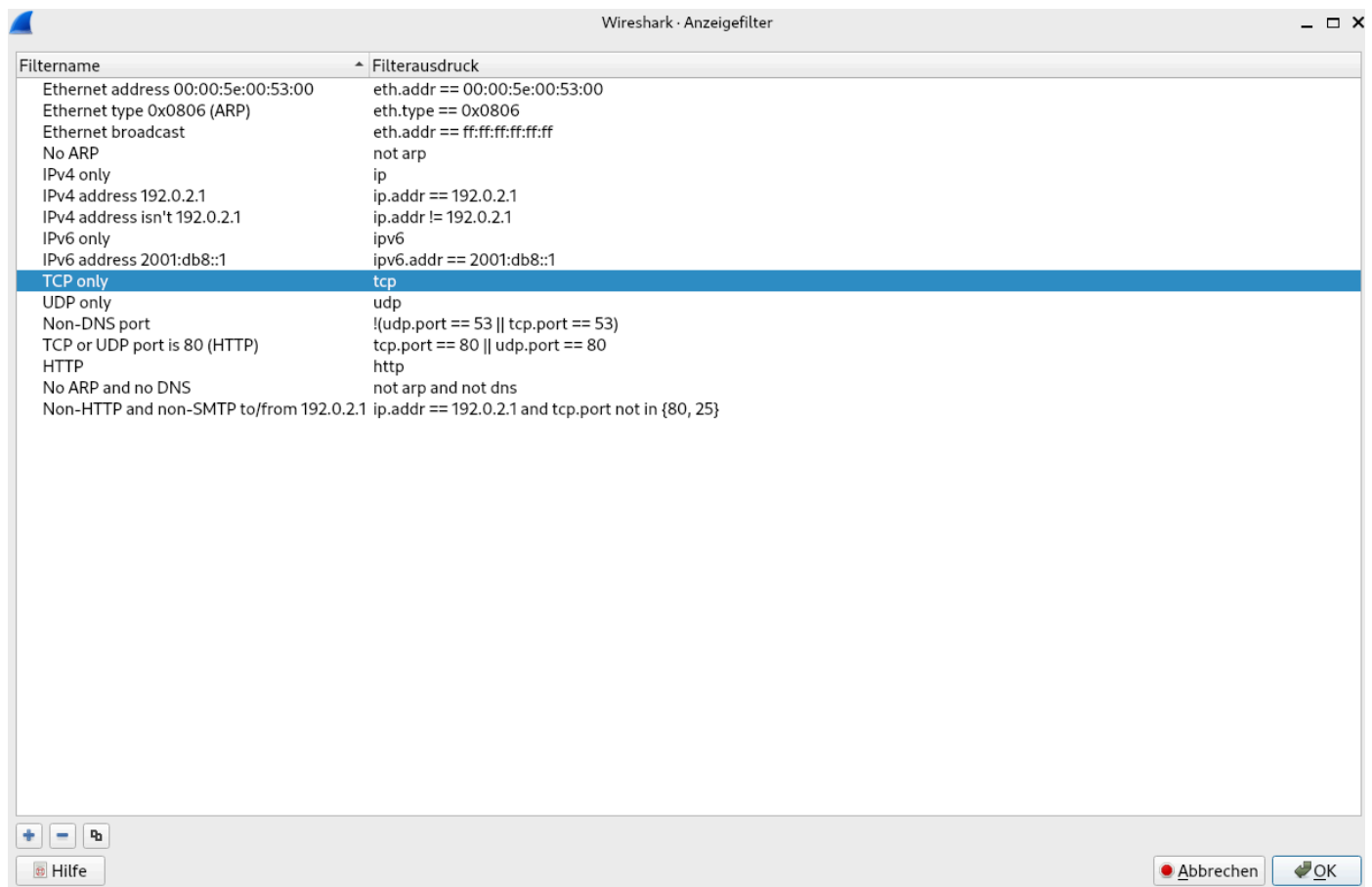


Abbildung 2. Anzeigefilter

Anzeigefilter werden nach der Aufzeichnung angewendet und bestimmen, welche Pakete in der Benutzeroberfläche angezeigt werden. Diese Filter ermöglichen es, die Ansicht zu verfeinern und nur die relevanten Pakete anzuzeigen. Beispiele für Anzeigefilter sind zum Beispiel bestimmte Protokolle, IP-Adressen oder Ports.

ngrep

Bei `ngrep` (Network grep) handelt es sich um ein Tool, das ähnlich wie das bekannte `grep` funktioniert, jedoch speziell für die Analyse von Netzwerkverkehr entwickelt wurde. Mit `ngrep` können Benutzer nach bestimmten Mustern in den Datenpaketen suchen, die über das Netzwerk übertragen werden. Dies ist besonders nützlich für die Fehlersuche, Sicherheitsüberwachung und Analyse von Netzwerkprotokollen. `ngrep` unterstützt eine Vielzahl von Protokollen, darunter TCP, UDP und ICMP, und ermöglicht es, sowohl den Header als auch den Payload der Pakete zu durchsuchen.

`ngrep` muss eventuell mit `apt install ngrep` nachinstalliert werden. Die generelle Syntax bei `ngrep` ist:

```
ngrep <Pattern> -d <Interface>
```

TERMINAL

Ein Beispiel hierfür könnte folgendermaßen aussehen:

```
ngrep GET -d enp0s3
```

TERMINAL

Im Beispiel wird nach dem Muster `GET` im Netzwerkverkehr auf der Schnittstelle `enp0s3` gesucht. Dies ist besonders nützlich, um HTTP-Anfragen zu überwachen, da `GET` eine häufig verwendete Methode in HTTP ist.

Verbindungen aufbauen

nc , telnet und ngrep

nc (Netcat) ist ein vielseitiges Tool, das für verschiedene Zwecke verwendet werden kann, darunter das Erstellen von Verbindungen auf Basis von TCP oder UDP, das Senden und Empfangen von Daten sowie das Debuggen und Testen von Netzwerkdiensten. Es wird oft als "Schweizer Taschenmesser" für Netzwerkanwendungen bezeichnet, da es eine breite Palette von Funktionen bietet. nc kann sowohl als Client als auch Server betrieben werden, was es zu einem nützlichen Werkzeug für Netzwerkadministratoren und Entwickler macht. Es gibt hierbei die verschiedensten Anwendungsfälle:

```
nc -l -p 12345
```

TERMINAL

Mit Hilfe dieses Befehls wird ein einfacher Server gestartet (-l steht für listen), der auf Port 12345 (-p 12345) auf eingehende Verbindungen wartet. Dieser Befehl kann jetzt zum Beispiel auf <User>_debian_server ausgeführt werden!

telnet ist ein Netzwerkprotokoll, das verwendet wird, um eine Verbindung zu einem entfernten Host über das TCP/IP-Protokoll herzustellen. Es ermöglicht Benutzern, sich auf einem entfernten Computer anzumelden und Befehle auszuführen, als ob sie direkt vor dem Computer sitzen würden. telnet wird häufig für die Fernverwaltung von Servern und Netzwerkgeräten verwendet. Allerdings ist telnet nicht verschlüsselt, was bedeutet, dass alle übertragenen Daten, einschließlich Passwörter, im Klartext gesendet werden. Aus diesem Grund wird telnet heutzutage oft durch sicherere Protokolle wie SSH (Secure Shell) ersetzt. Auf dem Rechner <User>_debian_client kann telnet zum Beispiel verwendet werden, um eine Verbindung zu dem zuvor gestarteten nc Server auf <User>_debian_server herzustellen:

```
telnet <IP-Adresse> 12345
```

TERMINAL

Um nun die Befehlshistorie zwischen den beiden beteiligten Rechnern einsehbar zu machen, kann auf dem nc Server mit Hilfe von ngrep die Eingabe sichtbar gemacht werden:

```
ngrep -d <Interface> . port 12345
```

TERMINAL

Der . steht hierbei für ein beliebiges Zeichen, also quasi "alles".

Man könnte hier jetzt auch einen kompletten Chat daraus basteln! Hierzu verwendet man auf dem Client nicht den telnet Befehl, sondern ebenfalls nc :

```
nc <IP-Adresse> 12345
```

TERMINAL

Auch das Übertragen von Dateien ist durch nc möglich! Hierzu wird auf dem Server eine Datei bereitgestellt:

```
cat example.txt | nc -l -p 12345
```

TERMINAL

Auf dem Client wird die Datei empfangen und in eine Datei geschrieben:

```
nc <IP-Adresse> 12345 > received_example.txt
```

TERMINAL

Weitere wichtige Parameter im Zusammenhang mit nc sind:

- **-v** : Aktiviert den `verbose` Modus, der detailliertere Informationen über die Verbindung und den Datenverkehr anzeigt.
- **-z** : Ermöglicht das Scannen von Ports, ohne tatsächlich Daten zu senden. Dies ist nützlich, um festzustellen, welche Ports auf einem Zielhost offen sind.

ARP-Spoofing

ARP-Spoofing ist eine Man-in-the-Middle-Attacke mit dem Ziel, den Datenverkehr zwischen zwei Geräten im Netzwerk abzufangen und möglicherweise zu manipulieren. Dabei wird die Tabelle für ARP (Address Resolution Protocol) eines Geräts gezielt mit falschen MAC-Adressen gefüllt, um den Datenverkehr umzuleiten. (vgl. [WIK2])

Die Beteiligten

Generell gibt es bei diesem Angriff drei verschiedene Rollen:

- **Das Opfer:**
Hierbei handelt es sich um einen User, der sich zum Beispiel an einem Computer im Netzwerk mit dessen Benutzernamen und Passwort anmeldet.
- **Das Ziel:**
Der eigentliche Server, an dem sich das Opfer mit Benutzernamen und Passwort anmeldet.
- **Der Angreifer:**
Es handelt sich um eine Person, die versucht, sich unbefugt Zugang zu den Daten des Opfers zu verschaffen, indem sie sich zwischen das Opfer und das Ziel schaltet. Folglich ist der Angreifer der **Man-in-the-Middle**.

Technischer Hintergrund

Wie bereits erwähnt, wird innerhalb eines Netzwerks das ARP-Protokoll verwendet, damit zu einer IP-Adresse die entsprechende MAC-Adresse gefunden werden kann. Hierfür wird ein ARP-Request als Broadcast an alle entsprechenden Geräte innerhalb des Netzwerks gesendet. Das Gerät, welches die angefragte IP-Adresse besitzt, antwortet mit einem ARP-Reply, in dem die zugehörige MAC-Adresse übermittelt wird. Diese Information wird in der ARP-Tabelle des anfragenden Geräts gespeichert, damit bei zukünftigen Anfragen nicht erneut ein ARP-Request gesendet werden muss. Das Problem bei ARP ist allerdings, dass es nicht authentifiziert, wer den ARP-Request schickt und wer den ARP-Reply gibt. Das bedeutet, dass eigentlich jeder im Netz antworten dürfte bzw. auch kann. Daher ist es möglich, dass ein Angreifer gefälschte ARP-Responses schicken kann:

- An das Ziel: "Die IP des Opfers gehört zu meiner MAC!"
- An das Opfer: "Die IP des Ziels gehört zu meiner MAC!"

Mit diesem Prinzip werden alle Pakete an den Angreifer geleitet. Damit Ziel und Opfer natürlich nichts bemerken, leitet der Angreifer die Pakete auch entsprechend weiter.

Das Szenario

Ziel des Angreifers ist es, dass zum Beispiel ein Username und ein Passwort ausgespäht werden können. Um das Prinzip zu testen, wird ein FTP-Server verwendet.

Schritte für das Ziel

Das Opfer versucht sich dabei mit Benutzernamen und Passwort am FTP-Server anzumelden. `<User>_debian_server` fungiert hierfür als FTP-Server. Dieser muss zunächst installiert werden:

```
apt install ftpd
```

TERMINAL

Schritte für das Opfer

Damit man ein ordentliches Opfer hat, benötigt man eine neue virtuelle Maschine. Diese Maschine ist ebenfalls im gleichen Netz wie auch der FTP-Server und der Angreifer. Die neue virtuelle Maschine trägt den Namen `<User>_debian_client2` und muss natürlich in das Netz eingehängt werden, damit die Rechner untereinander kommunizieren können. Sobald das erledigt ist und die Maschine als Opfer fungiert, benötigt man für den Rechner einen FTP-Client:

```
apt install ftp
```

TERMINAL

Jetzt kann man sich mit nachfolgendem Kommando auf den FTP-Server verbinden:

```
ftp <User>@<IP-Adresse>
```

TERMINAL

Um die Verbindung wieder zu beenden, reicht einer der nachfolgenden Kommandos:

- `bye`
- `quit`

Schritte für den Angreifer

Der Angreifer `<User>_debian_client` braucht zwei Tools, um den Angriff durchzuführen:

1. `arp spoof` : Dieses Tool wird verwendet, um gefälschte ARP-Nachrichten zu senden. Das Tool ist im Paket `dsniff` enthalten.
2. `tcpdump` : Dieses Tool wird verwendet, um den Netzwerkverkehr zu überwachen und die gestohlenen Anmeldeinformationen aufzuzeichnen (bereits bekannt).

IMPORTANT

`dsniff` beinhaltet verschiedene Tools, deren Verwendung grundlegend illegal ist, wenn das Opfer nicht freiwillig mitmacht!

Die Toolings müssen zunächst beim Angreifer installiert werden:

```
apt install dsniff  
apt install tcpdump
```

TERMINAL

Neben der Installation von Paketen, muss der Angreifer auch die Weiterleitung von Paketen aktivieren. Dies geschieht - wie bereits bekannt - über die Datei `/etc/sysctl.conf` :

```
net.ipv4.ip_forward=1
```

TERMINAL

Der Angreifer muss jetzt dem Netz folgende Informationen mitteilen:

1. Dem Ziel teilt er mit: "Ich bin das Opfer!"
2. Dem Opfer teilt er mit: "Ich bin das Ziel!"

Das passiert mit nachfolgendem Kommando:

```
arp spoof -i <Interface> -t <IP> <IP>
```

TERMINAL

Weiterhin muss der Datenverkehr protokolliert werden. Hierfür kommt `tcpdump` zum Einsatz:

```
tcpdump -lni <Interface> -s0 -w <NameOfFile> port 21
```

TERMINAL

In Wireshark kann im Anschluss die `*.pcap` Datei ausgewertet werden und das Passwort ist im Klartext ersichtlich.